

**LAPORAN PRAKTIKUM**  
**SISTEM ADMINISTRASI DAN INFORMASI TERDISTRIBUSI**  
**PERTEMUAN 12**  
**CI/CD PIPELINE LOKAL DENGAN GITEA ACTIONS**



Disusun oleh:

Nama : Hanan Fijananto  
NIM : 24/538946/SV/24555  
Kelas : B2  
Dosen Pengampu : Dr.Imam Fahrurrozi, S.T., M.Cs.

**PROGRAM STUDI D-IV TEKNOLOGI REKAYASA PERANGKAT**  
**LUNAK**  
**DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA**  
**SEKOLAH VOKASI**  
**UNIVERSITAS GADJAH MADA**  
**YOGYAKARTA**

**2026**

# Daftar Isi

|                                                               |           |
|---------------------------------------------------------------|-----------|
| <b>A Tujuan</b>                                               | <b>2</b>  |
| <b>B Dasar Teori</b>                                          | <b>2</b>  |
| B.1 CI/CD . . . . .                                           | 2         |
| B.2 git dan gitea . . . . .                                   | 3         |
| <b>C Hasil dan Pembahasan</b>                                 | <b>4</b>  |
| C.1 Persiapan Infrastruktur . . . . .                         | 4         |
| C.2 Persiapan objek aplikasi . . . . .                        | 5         |
| C.3 Instalasi Gitea menggunakan Docker Compose . . . . .      | 7         |
| C.4 Clone Repository . . . . .                                | 11        |
| C.5 Push aplikasi . . . . .                                   | 14        |
| C.6 Cek action CICD di aplikasi . . . . .                     | 15        |
| <b>D Tugas dan Analisis</b>                                   | <b>16</b> |
| D.0.1 Implementasi file test dan pembaruan workflow . . . . . | 16        |
| D.0.2 Analisis Skenario kegagalan pipeline . . . . .          | 18        |
| <b>E Kesimpulan</b>                                           | <b>20</b> |
| <b>F Daftar Pustaka</b>                                       | <b>21</b> |

## A Tujuan

1. Memahami konsep dasar CI/CD (Continuous Integration & Continuous Deployment) dalam pengembangan perangkat lunak modern
2. Mengimplementasikan otomatisasi deployment menggunakan Gitea Actions
3. Membuat script workflow .yaml untuk memerintahkan server melakukan update aplikasi secara otomatis via SSH
4. Membangun infrastruktur CI/CD mandiri menggunakan Gitea dan Gitea Runner

## B Dasar Teori

### B.1 CI/CD

Continuous integration (CI) adalah pengintegrasian kode ke dalam repositori kode kemudian menjalankan pengujian secara otomatis, cepat, dan sering. Continuous Integration dapat dilakukan dengan menggunakan perintah commit.

Sementara continuous delivery atau continuous deployment (CD) adalah praktik yang dilakukan setelah proses CI selesai dan seluruh kode berhasil terintegrasi, sehingga aplikasi bisa dibangun lalu dirilis secara otomatis.

CI/CD pipeline ini biasa digunakan dalam pengembangan perangkat lunak. CI/CD pipeline ini menjadi penghubung antara tim pengembang dengan tim operasional yang di dalamnya terdapat tiga fase yang berupa continuous integration, continuous delivery, dan continuous deployment. Ketiga fase tersebut akan dilakukan secara terus menerus dan otomatis untuk mendapatkan perangkat lunak yang andal dan bebas dari bug [1].

## B.2 git dan gitea

Git adalah sistem kontrol versi terdistribusi yang digunakan untuk melacak perubahan dalam kode sumber selama pengembangan perangkat lunak. Git memungkinkan banyak pengembang untuk bekerja pada proyek yang sama secara bersamaan tanpa konflik, dengan menyimpan riwayat perubahan dan memungkinkan kolaborasi yang efisien [2].

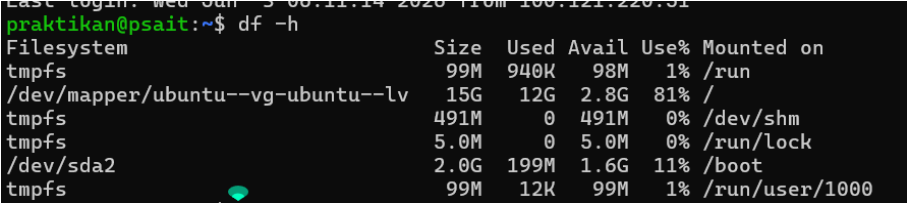
Gitea adalah platform Git repository hosting berbasis open source yang dirancang untuk dijalankan secara self hosted. Artinya, Gitea diinstal dan dijalankan di server milik sendiri, seperti Cloud VPS. Semua data, konfigurasi, dan akses pengguna sepenuhnya berada di bawah kendali pemilik server. Gitea menyediakan fitur-fitur seperti manajemen repositori, kolaborasi tim, issue tracking, pull request, dan integrasi dengan layanan lain. Dengan menggunakan Gitea, pengguna dapat mengelola proyek Git mereka dengan mudah tanpa harus bergantung pada layanan hosting Git pihak ketiga [3].

## C Hasil dan Pembahasan

### C.1 Persiapan Infrastruktur

Pastikan tempat penyimpanan dan ram dari vps masih tersedia. Untuk mengecek sisa penyimpanan dan ram, dapat menggunakan perintah berikut:

```
1 df -h
```



```
Last login: Wed Jan 3 08:11:14 2020 from 100.121.220.31
praktikan@psait:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           99M   940K   98M   1% /run
/dev/mapper/ubuntu--vg-ubuntu--lv 15G   12G   2.8G  81% /
tmpfs           491M    0   491M   0% /dev/shm
tmpfs           5.0M    0   5.0M   0% /run/lock
/dev/sda2       2.0G  199M   1.6G  11% /boot
tmpfs           99M   12K   99M   1% /run/user/1000
```

Gambar 1: Hasil perintah `df -h` untuk mengecek sisa penyimpanan

Berdasarkan gambar 1, sisa penyimpanan kurang cukup untuk menginstal Gitea dan Gitea Runner. Oleh karena itu, perlu dilakukan pembersihan image atau container yang tidak diperlukan untuk memberikan ruang yang cukup bagi instalasi Gitea dan Gitea Runner.

```

praktikan@psait:~$ docker stop 46d5d635f469
46d5d635f469
praktikan@psait:~$ docker stop f7773a835bc4
f7773a835bc4
praktikan@psait:~$ docker images

```

| IMAGE                         | ID            | DISK USAGE | CONTENT SIZE | EXTRA |
|-------------------------------|---------------|------------|--------------|-------|
| hello-world:latest            | 0e760fd9fbc48 | 25.9kB     | 9.49kB       | U     |
| jwt-php:latest                | 5000fd897740  | 708MB      | 176MB        |       |
| mysql:8.0                     | 7dcddc01f13b  | 1.1GB      | 249MB        | U     |
| php-jwt-example-be-jwt:latest | 8fe9aa7aaf04  | 708MB      | 176MB        |       |
| phpmyadmin:latest             | 85a2b4eb2da6  | 821MB      | 197MB        | U     |

```

praktikan@psait:~$ docker system prune -a
WARNING! This will remove:
- all stopped containers
- all networks not used by at least one container
- all images without at least one container associated to them
- all build cache

Are you sure you want to continue? [y/N] y
Deleted Containers:
46d5d635f469fba235840d6d83c429dbb9b6e8d02dbc46676a8e735af7a1a300
f7773a835bc4bb4964515387b73de0d20447585601210ce383de4ba7c2fb75bb
62ce8b1907365398cb3182ca81851965b126eed279c3b2297e7cae409676d4f3
082c2a5d998babf71f04f7d4e10d58ba591c9c79d22f8b4b4c4f4cdb388107c1

Deleted Networks:
php-jwt-example_psait

Deleted Images:
untagged: php-jwt-example-be-jwt:latest
deleted: sha256:8fe9aa7aaf0479f9c574cc6826787e9485e80267b30a7b9932d8c7f230b95990
deleted: sha256:4067898c18ad03a66536acf77ef92cb6c8705c9a00e765d6910e5dcf67f75368
deleted: sha256:1719e6dae8adb18444d074aef62088349affd14e03cea74d8b60b60f543b630f
untagged: phpmyadmin:latest

```

Gambar 2: Hasil perintah docker system prune untuk membersihkan image dan container yang tidak diperlukan

Setelah menghentikan container tersebut, hapus container yang tidak terpakai dengan perintah berikut:

```

1 docker system prune -a

```

## C.2 Persiapan objek aplikasi

Gunakan php-jwt-example yang telah dibuat pada pertemuan sebelumnya. Namun ubah kode docker-compose.yml untuk menyesuaikan dengan kebutuhan aplikasi. Matikan service database mysql di be-jwt (dapat dilihat pada gambar ??), dan ubah file .env sesuai dengan koneksi ke Mysql.

```
docker-compose.yml U X
1  services:
2      be-jwt:
3          ports:
4              - "8080:8080"
5          volumes:
6              - ./var/www/html
7          env_file:
8              - .env
9          restart: unless-stopped
10         #Network: agar rapi, ditaruh di network khusus
11         networks:
12             - psait
13         # depends_on:
14         #     - database_mysql
15
16         # database_mysql:
17         #     image: mysql:8.0
18         #     container_name: db_mysql_jwt
19         #     restart: unless-stopped
20         #     networks:
21         #         - psait
22         #     volumes:
23         #         - db_data:/var/lib/mysql
24         #         - ./php_jwt_db.sql:/docker-entrypoint-initdb.d/init.sql
25
26         # environment:
27         #     MYSQL_DATABASE: ${DB_NAME}
28         #     MYSQL_USER: ${DB_USER}
29         #     MYSQL_PASSWORD: ${DB_PASSWORD}
30         #     MYSQL_ALLOW_EMPTY_PASSWORD: "yes"
31         #     MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASS:-""}
32
33     phpmyadmin:
34         image: phpmyadmin:latest
```

Gambar 3: File docker-compose.yml yang telah diubah untuk menyesuaikan dengan kebutuhan aplikasi

```
1  DB_HOST=http://10.33.35.71
2  DB_PORT=3306
3  DB_DATABASE=php_jwt_db
4  DB_USERNAME=php_jwt
5  DB_PASSWORD=password123
```

Pastikan bahwa sistem service mysql di vps masih menyala dengan baik. Jika tidak menyala, maka jalankan perintah berikut untuk menyalakan kembali service mysql:

```
1  sudo systemctl start mysql
```

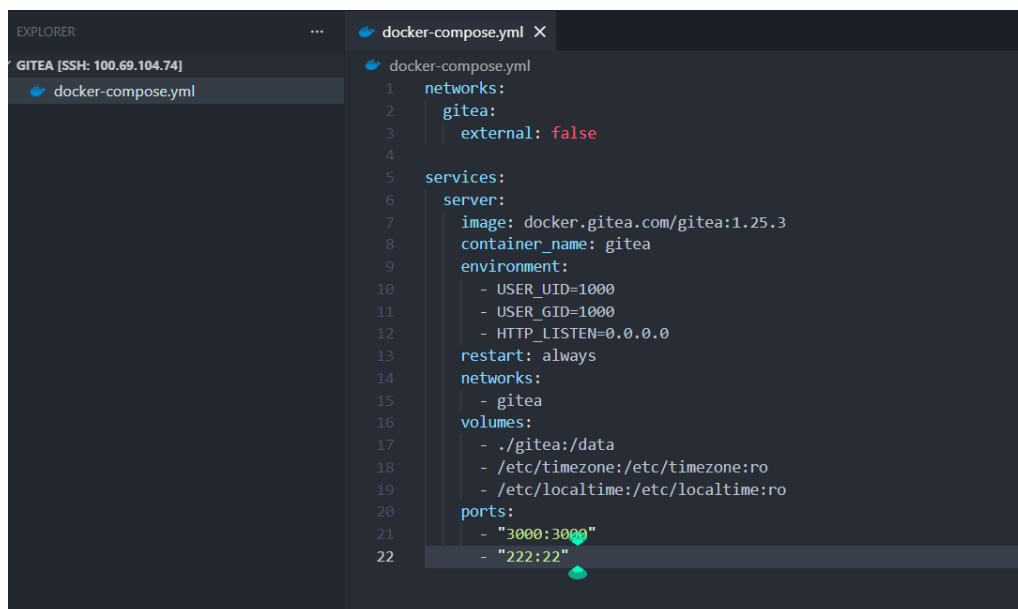
### C.3 Instalasi Gitea menggunakan Docker Compose

Buat folder bernama gitea di home direktori

```
praktikan@psait:~$ mkdir gitea
praktikan@psait:~$ cd gitea
praktikan@psait:~/gitea$
```

Gambar 4: Perintah untuk membuat folder gitea di home direktori

Di dalam folder tersebut, buat file bernama docker-compose.yml untuk proses instalasi gitea. Berikut adalah isi dari file docker-compose.yml untuk instalasi gitea:



```
1 networks:
2   gitea:
3     external: false
4
5 services:
6   server:
7     image: docker.gitea.com/gitea:1.25.3
8     container_name: gitea
9     environment:
10      - USER_UID=1000
11      - USER_GID=1000
12      - HTTP_LISTEN=0.0.0.0
13     restart: always
14     networks:
15       - gitea
16     volumes:
17       - ./gitea:/data
18       - /etc/timezone:/etc/timezone:ro
19       - /etc/localtime:/etc/localtime:ro
20     ports:
21       - "3000:3000"
22       - "222:22"
```

Gambar 5: Isi file docker-compose.yml untuk instalasi gitea

File `docker-compose.yml` tersebut digunakan untuk menjalankan layanan Gitea dalam sebuah kontainer Docker dengan image `docker.gitea.com/gitea:1.25.3`. Konfigurasi tersebut mengatur lingkungan eksekusi, penyimpanan data persisten pada direktori `./gitea/data`, serta menghubungkan kontainer ke jaringan Docker bernama `gitea`. Port 3000 diekspos untuk akses antarmuka web Gitea, sedangkan port 22 dipetakan untuk layanan SSH yang digunakan dalam operasi Git.

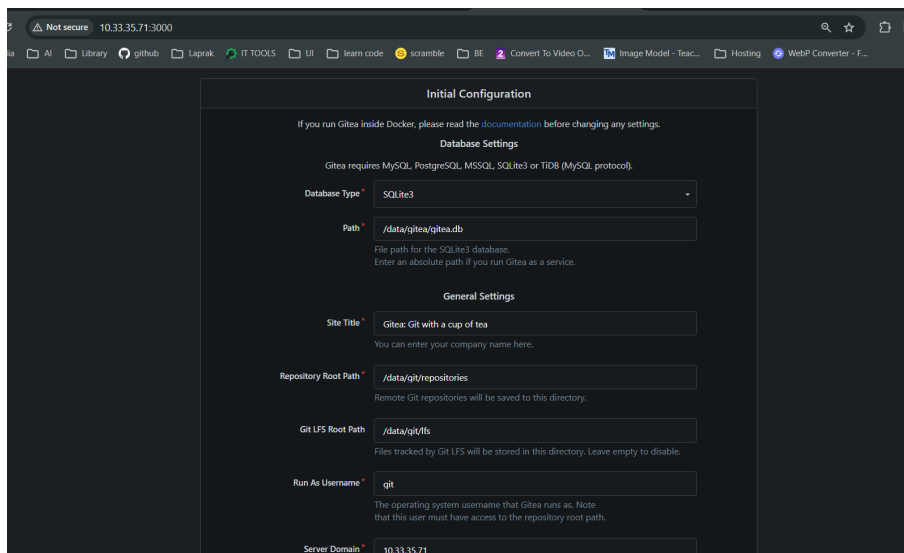
Jalankan service tersebut dengan perintah berikut



```
praktikan@psait:~/gitea$ docker compose up -d
[+] up 10/10
✓ Image docker.gitea.com/gitea:1.25.3 Pulled
✓ Network gitea_gitea Created
✓ Container gitea Started
```

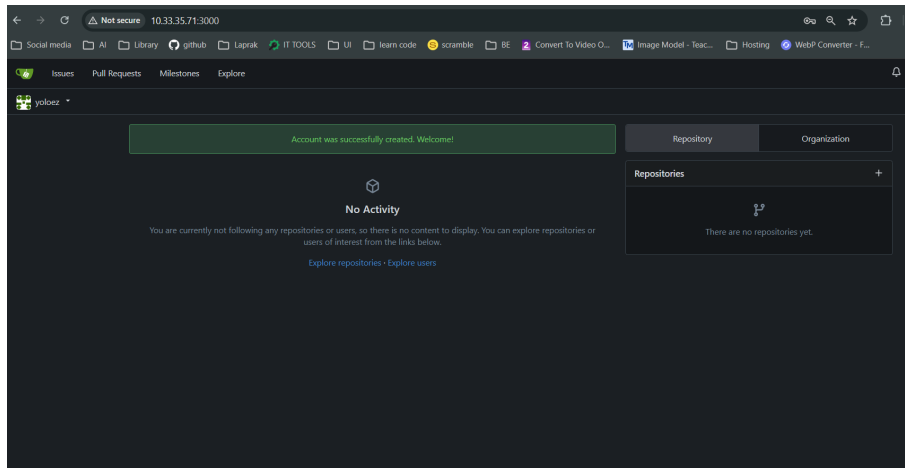
Gambar 6: Perintah untuk menjalankan service Gitea menggunakan Docker Compose

Setelah menjalankan perintah tersebut, tunggu beberapa saat hingga service Gitea berjalan dengan baik. Setelah itu, akses Gitea melalui browser dengan alamat `http://10.33.35.71:3000`. Pada halaman pertama, abaikan perubahan konfigurasi dan klik install.



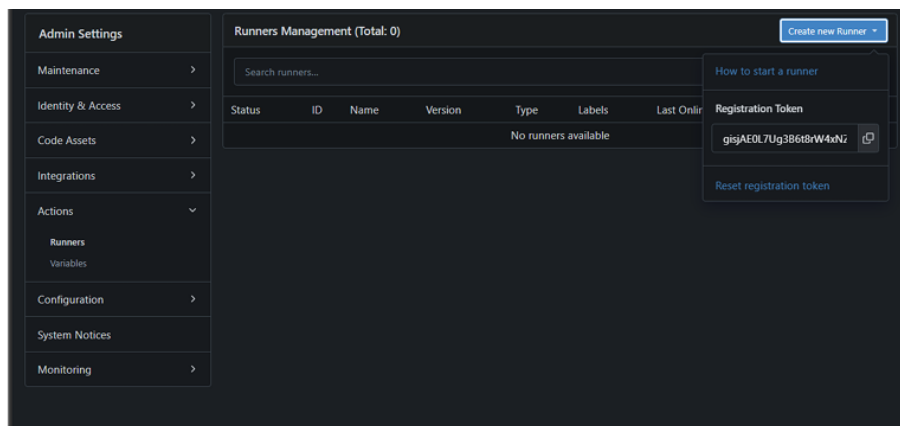
Gambar 7: Halaman instalasi Gitea

selanjutnya dapat langsung membuat akun git baru dan melakukan konfigurasi di laptop praktikan. Setelah melakukan registrasi, akan terlihat halaman dashboard Gitea seperti pada gambar 8.



Gambar 8: Halaman dashboard Gitea setelah berhasil melakukan registrasi akun

Pada tahap selanjutnya, buat runner agar dapat menjalankan cicd di gitea. Buka menu site administration, lalu pilih side bar action. Di situ terdapat registration token yang dapat digunakan untuk mendaftarkan runner.



Gambar 9: Menu untuk membuat runner di Gitea

Salin token tersebut, lalu paste di file docker-compose.yml untuk konfigurasi gitea runner. Tambahkan juga beberapa environment juga seperti pada gambar 10.

```
... docker-compose.yml X
5 services:
6   server:
16     volumes:
18       - /etc/timezone:/etc/timezone:ro
19       - /etc/localtime:/etc/localtime:ro
20     ports:
21       - "3000:3000"
22       - "222:22"
23
24   runner:
25     image: gitea/act_runner:latest
26     container_name: gitea_runner
27     restart: unless-stopped
28     networks:
29       - gitea
30     depends_on:
31       - server
32     volumes:
33       - /var/run/docker.sock:/var/run/docker.sock
34       - ./runner-data:/data
35     environment:
36       # URL Gitea (Gunakan nama service 'gitea' karena satu jaringan)
37       - GITEA_INSTANCE_URL=http://gitea:3000
38       # PASTE TOKEN DARI LANGKAH 1 DI SINI
39       -
40       - GITEA_RUNNER_REGISTRATION_TOKEN=Cop5etBv2TkjXdydsK9TKtEz6oULEkZQHbgACRT4
41       - GITEA_RUNNER_NAME=runner-1
42       # MAPPING LABEL (Sangat Penting untuk Storage Hemat!)
43       # Artinya: Jika ada job 'ubuntu-latest', pakai image 'node:16-alpine'
44       - GITEA_RUNNER_LABELS=ubuntu-latest:docker://node:16-alpine
45
```

Gambar 10: Konfigurasi file docker-compose.yml untuk gitea runner

Jalankan `docker compose up -d` kembali untuk menjalankan service gitea runner. Setelah itu, cek kembali di halaman action runner, maka runner yang telah dibuat akan muncul di sana.

Selanjutnya, buat repository baru di Gitea dengan nama `php-jwt-example` untuk menyimpan kode aplikasi yang telah dibuat sebelumnya. Sebelum melakukan commit, dan push kode, hapus terlebih dahulu remote asli agar tidak melakukan push ke repository aslinya. Gunakan perintah berikut untuk menghapus remote asli:

```
1 rm -rf .git
```

Setelah itu, pastikan tambahkan `.gitignore` untuk mengabaikan file `.env` agar tidak ikut terpush ke repository. Sebelum melakukan push, pastikan mendaftarkan git credential dengan email dan name terlebih dahulu dengan perintah berikut

```
praktikan@psait:~/psait/php-jwt-example$ git config --global user.email "hananfijananto@gmail.com"
praktikan@psait:~/psait/php-jwt-example$ git config --global user.name "yoloez"
```

Gambar 11: Perintah untuk mendaftarkan git credential dengan email dan name

```
praktikan@psait:~/psait/php-jwt-example$ git config --global user.email "hananfijananto@gmail.com"
praktikan@psait:~/psait/php-jwt-example$ git config --global user.name "yoloez"
praktikan@psait:~/psait/php-jwt-example$ git add .
praktikan@psait:~/psait/php-jwt-example$ git commit -m "first init"
[main (root-commit) 6e2e90b] first init
36 files changed, 3668 insertions(+)
create mode 100644 .gitignore
create mode 100644 .htaccess
create mode 100644 Dockerfile
create mode 100644 README.md
create mode 100644 api/login/index.php
create mode 100644 api/register/index.php
create mode 100644 api/users/index.php
create mode 100644 composer.json
create mode 100644 composer.lock
create mode 100644 config/database.php
create mode 100644 docker-compose.yml
create mode 100644 index.php
create mode 100644 php_jwt_db.sql
create mode 100644 vendor/autoload.php
create mode 100644 vendor/composer/ClassLoader.php
create mode 100644 vendor/composer/InstalledVersions.php
create mode 100644 vendor/composer/LICENSE
create mode 100644 vendor/composer/autoload_classmap.php
create mode 100644 vendor/composer/autoload_namespaces.php
create mode 100644 vendor/composer/autoload_psr4.php
create mode 100644 vendor/composer/autoload_real.php
create mode 100644 vendor/composer/autoload_static.php
```

Gambar 12: Perintah untuk melakukan push kode ke repository

## C.4 Clone Repository

Setelah melakukan push kode ke repository, maka kode tersebut akan tersimpan di repository Gitea. Clone repository tersebut ke dalam komputer lokal dengan perintah berikut:

```
1 git clone http://10.33.35.71:300/yoloez/php-jwt-example.git
```

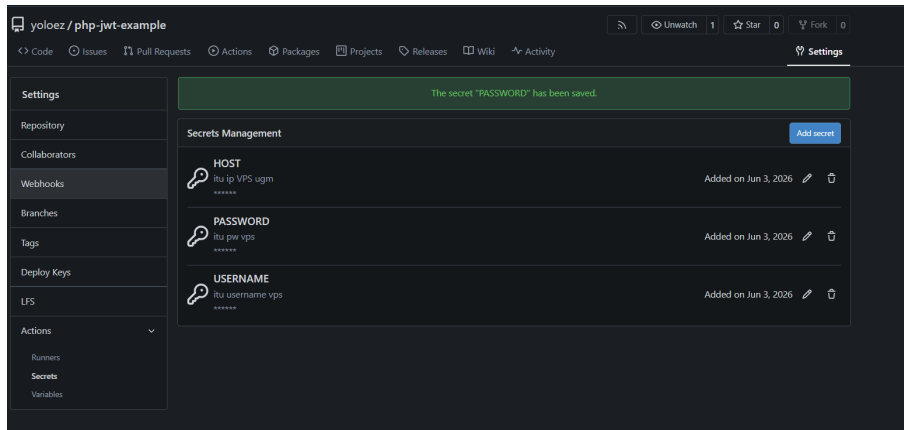
Masuk ke dalam projek tersebut di vscode lokal, lalu buat file deploys.yaml di dalam folder .gitea/workflows dengan isi sebagai berikut:

```
1 name: Auto Deploy via SSH Password
2 run-name: ${GITHUB_ACTOR} sedang deploy aplikasi
3
4 on:
5   push:
6     branches:
7       - main
8
9 jobs:
10  deploy-ke-server:
11    runs-on: ubuntu-latest
12
13    steps:
14      - name: Menghubungkan ke Server via SSH
15        uses: appleboy/ssh-action@1.0.0
16        with:
17          host: ${ secrets.HOST }
18          username: ${ secrets.USERNAME }
19          password: ${ secrets.PASSWORD }
20          port: 22
21
22      - name: Pre-cleanup: membersihkan sampah sebelum mulai...
23        run: |
24          #hapus image dan container yang sudah tidak terpakai
25          docker system prune -f
26
27      - name: Pulling latest code from repository...
28        run: |
29          cd psait/php-jwt-example
30          git pull origin main
31
32      - name: Building and deploy
33        run: |
34          #matikan container dulu
35          docker compose down
36
37          #buil baru dan jalankan container
38          docker compose up -d --build
39
40      - name: POST-CLEANUP: membersihkan sampah setelah deploy...
41        run: |
42          docker system prune -f --volumes
43
44      - name: cek sisa disk (untuk log history)
45        run: df -h | grep /dev/
```

Gambar 13: Isi file deploys.yaml untuk konfigurasi workflow Gitea Actions

File `deploys.yaml` tersebut digunakan untuk mendefinisikan workflow Gitea Actions yang akan dijalankan setiap kali terjadi push ke branch `main`. Workflow ini terdiri dari beberapa langkah, termasuk checkout kode, setup SSH, dan menjalankan perintah deployment melalui SSH ke server tujuan.

Selanjutnya, tambahkan secret variabel di gitea untuk menyimpan private key yang digunakan untuk koneksi SSH ke server. Private tersebut terdiri dari ip vps, username vps, dan password vps.



Gambar 14: Menu untuk menambahkan secret variabel di Gitea

Sebelum melakukan push kode, tambahkan endpoint `/api/cicd` untuk mengecek sebelum terjadi perubahan.

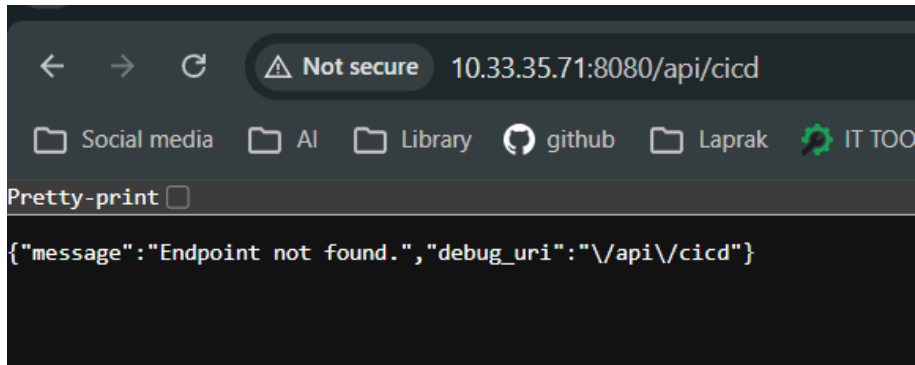
```

31         echo json_encode([ "message" => "File api/register/index.php tidak ditemukan!"]);
32     }
33     break;
34
35     case '/api/users':
36         if (file_exists('api/users/index.php')) {
37             require 'api/users/index.php';
38         } else {
39             http_response_code(500);
40             echo json_encode(["message" => "File api/users/index.php tidak ditemukan!"]);
41         }
42         break;
43
44     case '/api/cicd':
45         http_response_code(200);
46         echo json_encode(array("message" => "File api/cicd/index.php tidak ditemukan!"));
47         break;
48
49     case '/docker':
50         http_response_code(200);
51         echo json_encode(["message" => "Docker API endpoint is working!"]);
52         break;
53
54     case '/':
55     case '/index.php':
56         http_response_code(200);
57         echo json_encode(array("message" => "Welcome to the API service"));
58         break;
59
60     default:
61         http_response_code(404);

```

Gambar 15: Penambahan endpoint `/api/cicd` untuk mengecek sebelum terjadi perubahan

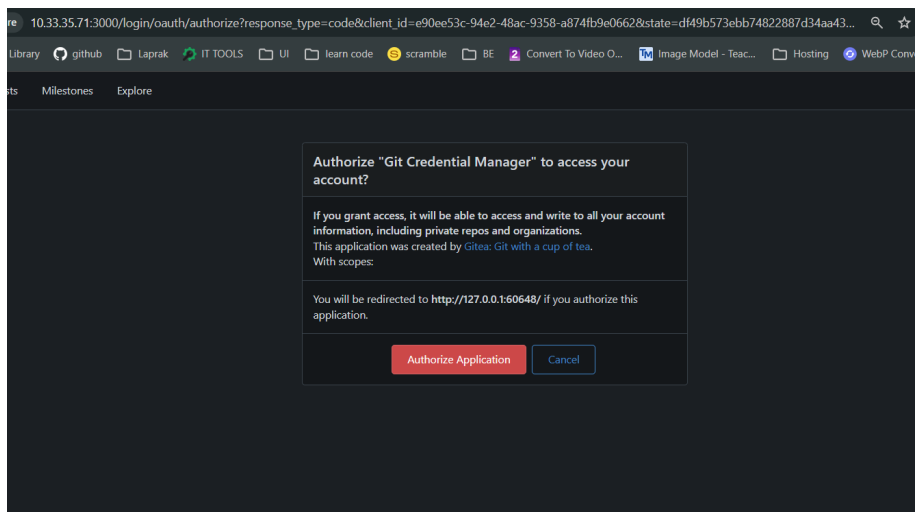
Akan muncul message seperti pada gambar 16 ketika endpoint tersebut diakses. Setelah itu, lakukan push kode ke repository untuk menjalankan workflow Gitea Actions.



Gambar 16: Response ketika endpoint /api/cicd diakses

## C.5 Push aplikasi

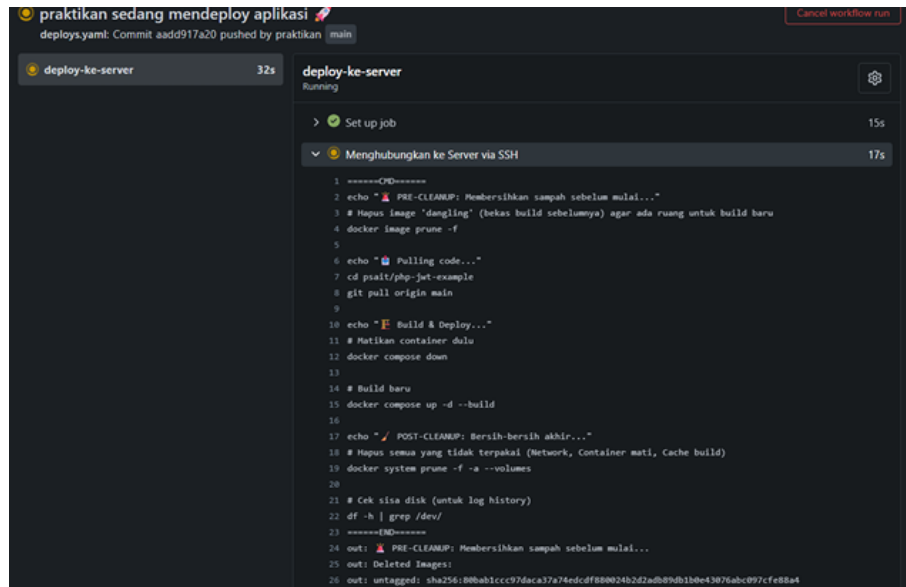
Masukkan semua perubahan ke staging area dengan `git add .`, lalu tambahkan commit message. Saat melakukan push, akan muncul tampilan seperti pada gambar 17 yang menunjukkan bahwa perlu melakukan authorization application terlebih dahulu. Klik authorize untuk memberikan akses ke Gitea Actions agar dapat menjalankan workflow yang telah dibuat sebelumnya.



Gambar 17: Tampilan ketika melakukan push kode ke repository Gitea

Setelah melakukan push, maka workflow Gitea Actions akan berjalan secara otomatis. Untuk melihat prosesnya, dapat membuka menu Actions di repository Gitea. Di sana akan terlihat bahwa workflow sedang berjalan dan menjalankan

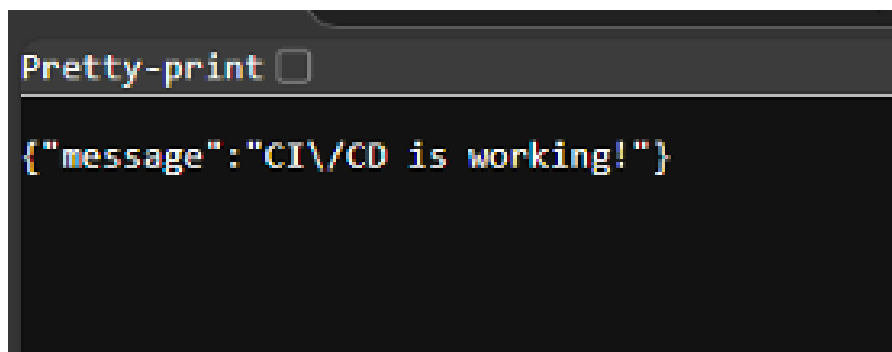
langkah-langkah yang telah didefinisikan di file `deploys.yaml`. Jika semua langkah berhasil dijalankan, maka aplikasi akan terdeploy secara otomatis ke server tujuan melalui SSH.



Gambar 18: Tampilan ketika workflow Gitea Actions sedang berjalan

## C.6 Cek action CI/CD di aplikasi

Buka kembali endpoint `/api/cicd` untuk mengecek apakah aplikasi sudah terdeploy dengan baik. Jika aplikasi sudah terdeploy, maka akan muncul response seperti pada gambar 19 yang menunjukkan bahwa aplikasi sudah berhasil diupdate dengan perubahan terbaru.



Gambar 19: Response ketika endpoint `/api/cicd` diakses setelah aplikasi terdeploy



## D Tugas dan Analisis

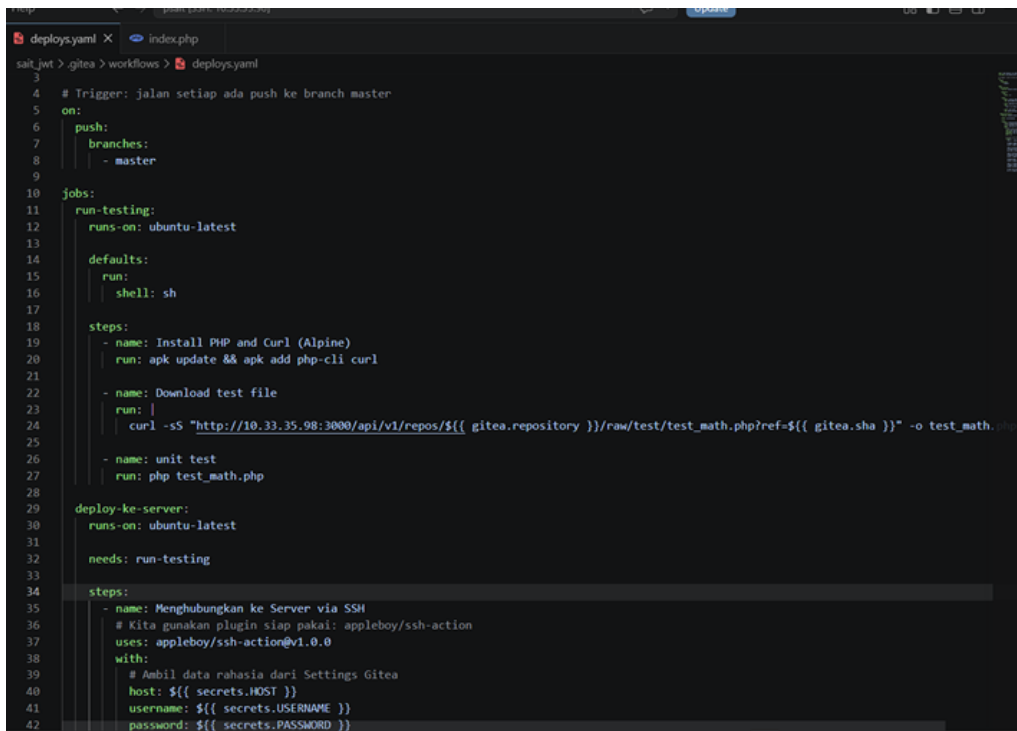
Mahasiswa diminta membuat folder baru bernama test dan tambahkan file bernama test\_matematika\_sederhana.php. Isi dari file tersebut yaitu

```
1      <?php
2      // Logika sederhana
3      $a = 5;
4      $b = 10;
5      $hasil = $a + $b;
6
7      echo "Menjalankan Test: $a + $b ... \n";
8
9      // Logika Validasi
10     if ($hasil == 15) {
11         echo "[SUKSES] Perhitungan benar. Silakan lanjut
12             deploy.\n";
13         exit(0); // Kode 0 artinya Aman/Sukses
14     } else {
15         echo "[GAGAL] Perhitungan salah! Jangan deploy ke
16             server!\n";
17         exit(1); // Kode 1 (atau selain 0) artinya Error/Stop
18     }
19     ?>
```

### D.0.1 Implementasi file test dan pembaruan workflow

Mahasiswa diminta menjalankan file test tersebut ke dalam Actions workflows yang telah kita dibuat. Modifikasi deploys.yaml agar runner yang ada di gitea mampu membatalkan aksi deploy jika logika sederhana tersebut salah! Screenshots juga detail workflow Actions yang ada di Gitea ketika sudah selesai berjalan!

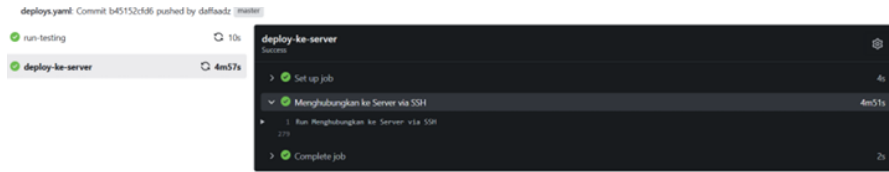
File `deploys.yaml` kemudian diperbarui untuk mengintegrasikan tahap pengujian. Job baru bernama `run-testing` ditambahkan dan dikonfigurasi untuk berjalan sebelum job `deploy-ke-server`. Dependency antar-job ini ditegaskan melalui direktif `needs: run-testing` pada job `deploy`. Di dalam job `run-testing`, langkah-langkahnya meliputi instalasi PHP dan curl pada runner, pengunduhan file test dari Gitea melalui API raw, dan eksekusi skrip test menggunakan perintah `php test_matematika_sederhana.php`.



```
3
4 # Trigger: jalan setiap ada push ke branch master
5 on:
6   push:
7     branches:
8       - master
9
10 jobs:
11   run-testing:
12     runs-on: ubuntu-latest
13
14     defaults:
15       run:
16         shell: sh
17
18     steps:
19       - name: Install PHP and Curl (Alpine)
20         run: apk update && apk add php-cli curl
21
22       - name: Download test file
23         run: |
24           curl -sS "http://10.33.35.98:3000/api/v1/repos/{{ gitea.repository }}/raw/test/test_math.php?ref={{ gitea.sha }}" -o test_math.php
25
26       - name: unit test
27         run: php test_math.php
28
29   deploy-ke-server:
30     runs-on: ubuntu-latest
31
32     needs: run-testing
33
34     steps:
35       - name: Menghubungkan ke Server via SSH
36         # Kita gunakan plugin siap pakai: appleboy/ssh-action
37         uses: appleboy/ssh-action@v1.0.0
38         with:
39           # Ambil data rahasia dari Settings Gitea
40           host: ${ secrets.HOST }
41           username: ${ secrets.USERNAME }
42           password: ${ secrets.PASSWORD }
```

Gambar 20: File `deploys.yaml` yang telah diperbarui untuk mengintegrasikan tahap pengujian

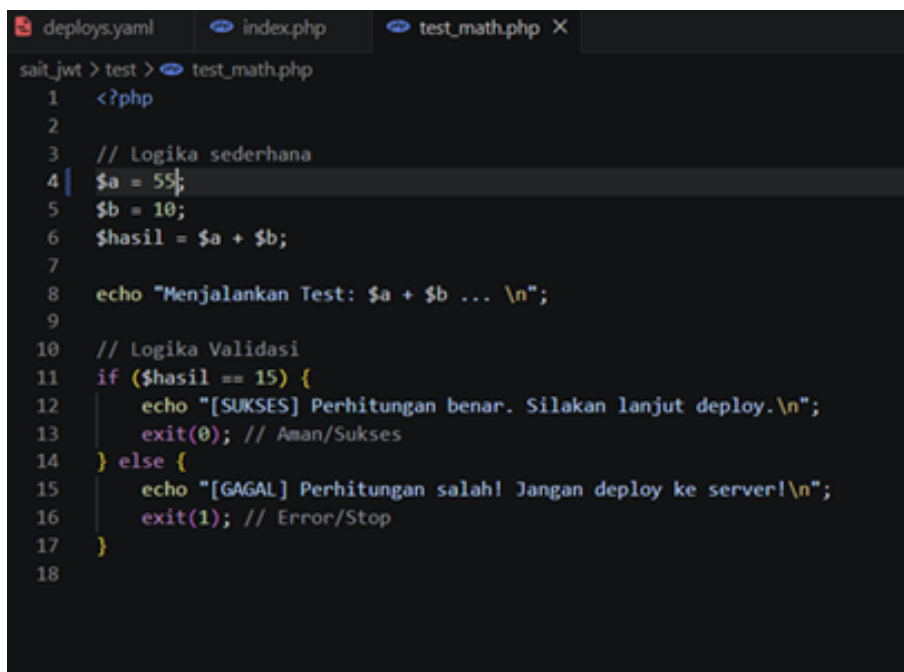
Setelah file test dan workflow yang diperbarui dipush ke repository Gitea, maka workflow akan berjalan secara otomatis. Hasil eksekusi pipeline dengan dua job ditampilkan pada Gambar 21. Kedua job, yaitu `run-testing` (10 detik) dan `deploy-ke-server` (4 menit 57 detik), berhasil diselesaikan dengan status sukses, yang membuktikan bahwa integrasi unit test ke dalam pipeline berjalan dengan benar



Gambar 21: Output hasil eksekusi pipeline dengan dua job di Gitea Actions

## D.0.2 Analisis Skenario kegagalan pipeline

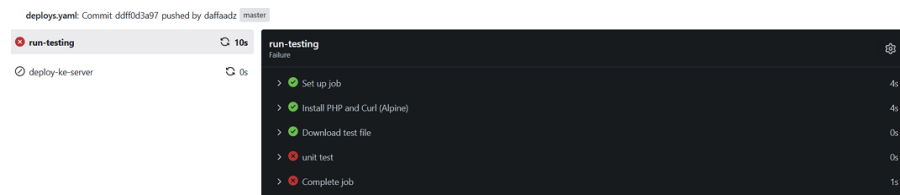
Untuk memahami mekanisme proteksi yang diberikan oleh unit test, dilakukan simulasi kegagalan dengan sengaja mengubah nilai variabel `$a` dari 5 menjadi 55 pada file `test_matematika_sederhana.php`.



Gambar 22: Merubah nilai variabel `$a` untuk mensimulasikan kegagalan pada unit test

Lakukan commit dan push perubahan tersebut ke repository Gitea, maka workflow akan berjalan secara otomatis. Hasil eksekusi pipeline dengan dua job ditampilkan pada Gambar 23. Job `run-testing` gagal dengan status `error`, yang menunjukkan bahwa unit test berhasil mendeteksi kesalahan logika dalam skrip test. Aki-

batnya, job deploy-ke-server tidak dijalankan, yang membuktikan bahwa mekanisme proteksi terhadap kesalahan telah berfungsi dengan baik.



Gambar 23: Pipeline job unit test gagal dan job deploy tidak dieksekusi

## E Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa implementasi CI/CD Pipeline menggunakan Gitea Actions berhasil diterapkan untuk mengotomatisasi proses integrasi dan deployment aplikasi. Infrastruktur yang dibangun mencakup instalasi Gitea, konfigurasi Gitea Runner, pengelolaan repositori, serta pembuatan workflow berbasis `deploys.yaml` yang dijalankan secara otomatis setiap kali terjadi perubahan pada branch `main`. Melalui mekanisme ini, proses deployment aplikasi ke server dapat dilakukan secara lebih cepat, konsisten, dan efisien tanpa memerlukan intervensi manual, sehingga mendukung praktik pengembangan perangkat lunak modern yang lebih andal.

Selain itu, integrasi tahap pengujian (testing) ke dalam pipeline membuktikan pentingnya validasi otomatis sebelum proses deployment dilakukan. Hasil pengujian menunjukkan bahwa ketika skrip uji berjalan dengan sukses, proses deployment dapat dilanjutkan dan aplikasi berhasil diperbarui pada server. Sebaliknya, ketika terjadi kesalahan logika pada skrip pengujian, workflow secara otomatis menghentikan proses deployment sehingga perubahan yang bermasalah tidak diterapkan ke lingkungan produksi. Dengan demikian, penggunaan Gitea Actions tidak hanya meningkatkan otomatisasi proses pengembangan, tetapi juga memberikan mekanisme proteksi yang efektif untuk menjaga kualitas dan stabilitas aplikasi.

## F Daftar Pustaka

- [1] R. Setiawan, “Apa Itu CI/CD? IT Developer Harus Tau - Dicoding Blog — dicoding.com,” <https://www.dicoding.com/blog/apa-itu-ci-cd/>, 2021, [Accessed 16-06-2026].
- [2] Prasatya, “Apa Itu Git? Panduan Lengkap untuk Pemula: Pengertian, Fungsi, dan Cara Kerjanya - CODEPOLITAN — codepolitan.com,” <https://www.codepolitan.com/blog/apa-itu-git-panduan-lengkap-untuk-pemula-pengertian-fungsi-dan-cara-kerjanya/>, 2025, [Accessed 16-06-2026].
- [3] plasawebhost.com, “Gitea vs GitHub: Mana yang Lebih Tepat untuk Kebutuhan Pengelolaan Source Code? — plasawebhost.com,” <https://plasawebhost.com/blog/gitea-vs-github-mana-yang-lebih-tepat-untuk-kebutuhan-pengelolaan-source-code.html>, 2026, [Accessed 16-06-2026].